

Reverse Coding Challenge Set

CNC Junior — Answer Key & Facilitator Guide

9 problems · sample I/O verified by execution · solution, approach & difficulty for each

How to use this set

Each problem below is designed as a 'reverse coding' exercise: students are given only the Test Case Description and the Sample Input / Sample Output, and must infer the underlying logic themselves before writing code. This document is the facilitator/answer-key version — it additionally includes the intended reasoning path (Observations), the Approach, a Difficulty rating, and verified working code for each problem.

When running this as an exercise, share only the “Test Case Description” and “Sample Input / Output” sections with students, and withhold the Observations, Approach, Difficulty and Code sections until after they attempt a solution.

Problem Overview

#	Title	Difficulty
1	Fibonacci Prefix Sum	Easy-Medium
2	Maximum Product of Three Numbers	Medium
3	Sum of Primes up to N	Easy-Medium
4	Binary Length (Number of Bits)	Easy
5	Sum of Two Squares	Medium-Hard
6	Longest Strictly Increasing Subsequence (LIS)	Hard
7	Count Palindromic Substrings	Hard
8	Last Row of Pascal's Triangle	Medium
9	Maximum Points on a Line	Hard

Question 1: Fibonacci Prefix Sum

Test Case Description

First line contains an integer T ($1 \leq T \leq 100$), the number of test cases.

Each of the next T lines contains an integer N , the input value for that test case.

Sample I/O

Sample Input	Sample Output
4	0
0	1
2	4
4	7
5	

Difficulty: **Easy-Medium**

Observations (how a student can reverse-engineer the logic)

- The output grows in a non-linear, accelerating way as N increases — a hint that it involves a recursive or additive sequence rather than a simple formula.
- $N = 0$ gives 0, which rules out anything based on N itself (like N^2 or 2^N) and suggests an accumulation that starts empty.
- Try writing out the first few Fibonacci numbers (0, 1, 1, 2, 3, 5...) and their running sums: sum of first 0 terms = 0, first 2 terms = $0+1 = 1$, first 4 terms = $0+1+1+2 = 4$, first 5 terms = $0+1+1+2+3 = 7$. This matches exactly.

Approach / Logic

The task is to output the sum of the first N terms of the Fibonacci series (0-indexed, starting 0, 1, 1, 2, 3...).

Maintain two running variables $a = 0$ and $b = 1$ representing consecutive Fibonacci numbers, loop N times adding the current value of a to a running sum, then advance a and b . Print the sum for each test case.

Solution Code (C++)

```
#include <iostream>
using namespace std;

int main() {
    int T;
    cin >> T;
    while (T--) {
        int N;
        cin >> N;
        int a = 0, b = 1;
        int sum = 0;
        for (int i = 0; i < N; i++) {
            sum += a;
            int c = a + b;
            a = b;
            b = c;
        }
        cout << sum << endl;
    }
}
```

```
}  
return 0;  
}
```

Question 2: Maximum Product of Three Numbers

Test Case Description

First line contains an integer T, the number of test cases.

For each test case, the first line contains N ($N \geq 3$), followed by N integers.

Sample I/O

Sample Input	Sample Output
3	24
4	315
1 2 3 4	4752
6	
9 3 5 7 1 4	
5	
11 6 8 54 2	

Difficulty: **Medium**

Observations (how a student can reverse-engineer the logic)

- For [1,2,3,4], the answer $24 = 2 \times 3 \times 4$, the three largest values — suggests picking the top three numbers.
- For [9,3,5,7,1,4] the three largest are 9,7,5 giving 315, which matches — confirms the 'largest three' hypothesis for this input.
- For [11,6,8,54,2], the three largest are 54,11,8 = 4752, which also matches, so no negative-number edge case is visible from these samples alone — a well-formed guess should still consider what happens with negative numbers (two very negative numbers multiplied together become a large positive).

Approach / Logic

Sort the array. The maximum product of three numbers is the larger of: (a) the product of the three largest values, or (b) the product of the two smallest (most negative) values with the single largest value — useful when the array can contain negative numbers. Although the sample data only exercises case (a), a robust solution checks both.

Solution Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<int> nums(n);
        for (int i = 0; i < n; i++) {
            cin >> nums[i];
        }
        sort(nums.begin(), nums.end());
        int product1 = nums[n - 1] * nums[n - 2] * nums[n - 3];
```

```
        int product2 = nums[0] * nums[1] * nums[n - 1];  
        cout << max(product1, product2) << endl;  
    }  
    return 0;  
}
```

Question 3: Sum of Primes up to N

Test Case Description

First line contains an integer T, the number of test cases.

The next T lines each contain one number between 1 and 100.

Sample I/O

Sample Input	Sample Output
3	2
2	41
13	41
15	

Difficulty: Easy-Medium

Observations (how a student can reverse-engineer the logic)

- $N = 2$ gives output 2 — exactly N itself, hinting 2 is being included and is prime.
- $N = 13$ gives 41. The primes ≤ 13 are 2, 3, 5, 7, 11, 13, which sum to exactly 41.
- $N = 15$ also gives 41, the same as $N = 13$, even though N changed — a strong clue that intermediate composite values (14, 15) don't change the answer, pointing to a prime-filtering + summation logic.

Approach / Logic

For each N , iterate through every integer from 2 to N , test each for primality with a simple $O(\sqrt{v_i})$ trial-division check, and accumulate the primes into a running sum. Print the sum for each test case.

Solution Code (C++)

```
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n < 2)
        return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0)
            return false;
    }
    return true;
}

int main() {
    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        int sum = 0;
        for (int i = 2; i <= n; i++) {
            if (isPrime(i))
                sum += i;
        }
    }
}
```

```
    }  
    cout << sum << endl;  
}  
return 0;  
}
```

Question 4: Binary Length (Number of Bits)

Test Case Description

First line contains an integer T ($1 \leq T \leq 100$), the number of test cases.

The next T lines contain an integer N , the input value for each test case.

Sample I/O

Sample Input	Sample Output
3	4
10	2
2	5
24	

Difficulty: **Easy**

Observations (how a student can reverse-engineer the logic)

- 10 in binary is 1010 — 4 digits, matching the output 4.
- 2 in binary is 10 — 2 digits, matching the output 2.
- 24 in binary is 11000 — 5 digits, matching the output 5. The pattern is clearly 'number of binary digits needed to represent N '.

Approach / Logic

Repeatedly divide N by 2 (integer division), counting how many divisions are needed until N becomes 0. This count equals the number of bits in N 's binary representation (equivalent to $\text{floor}(\log_2(N)) + 1$ for $N > 0$).

Solution Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        int len = 0;
        while (n > 0) {
            len++;
            n /= 2;
        }
        cout << len << endl;
    }
    return 0;
}
```

Question 5: Sum of Two Squares

Test Case Description

First line contains an integer T ($1 \leq T \leq 100$), the number of test cases.

The next T lines contain an integer N , the input value for each test case.

Sample I/O

Sample Input	Sample Output
3	True
5	True
89	False
28	

Difficulty: Medium-Hard

Observations (how a student can reverse-engineer the logic)

- Output is Boolean (True/False), so the task is a yes/no check rather than a computed value.
- $5 = 1^2 + 2^2$ ($1 + 4$) $\rightarrow$ True. $89 = 5^2 + 8^2$ ($25 + 64$) $\rightarrow$ True. Try all pairs for 28: no combination of squares sums to 28 $\rightarrow$ False.
- This points to: 'does there exist a pair of non-negative integers whose squares add up to C ?'

Approach / Logic

Use a two-pointer scan: start $a = 0$ and $b = \text{sqrt}(C)$. While $a \leq b$, compute $a^2 + b^2$. If it equals C , a valid pair exists (return True). If the sum is less than C , increase a ; if greater, decrease b . If the pointers cross without finding a match, return False. This runs in $O(\sqrt{C})$ instead of checking all pairs exhaustively.

Solution Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

bool judgeSquareSum(int c) {
    long long a = 0;
    long long b = sqrt(c);
    while (a <= b) {
        long long sum = a * a + b * b;
        if (sum == c)
            return true;
        else if (sum < c)
            a++;
        else
            b--;
    }
    return false;
}

int main() {
    int T;
    cin >> T;
    while (T--) {
```

```
    int c;  
    cin >> c;  
    if (judgeSquareSum(c))  
        cout << "True" << endl;  
    else  
        cout << "False" << endl;  
}  
return 0;  
}
```

Question 6: Longest Strictly Increasing Subsequence (LIS)

Test Case Description

First line contains an integer T ($1 \leq T \leq 100$), the number of test cases.

For each test case, the first line contains N , the number of elements, followed by a line of N integers.

Sample I/O

Sample Input	Sample Output
3	4
8	4
10 9 2 5 3 7 101 18	1
6	
0 1 0 3 2 3	
7	
7 7 7 7 7 7 7	

Difficulty: **Hard**

Observations (how a student can reverse-engineer the logic)

- $[10,9,2,5,3,7,101,18] \rightarrow 4$. One increasing run is 2,5,7,101 (or 2,3,7,101 / 2,3,7,18) — length 4, and no longer strictly increasing run exists.
- $[0,1,0,3,2,3] \rightarrow 4$, from 0,1,2,3 (picked non-contiguously) — confirms 'subsequence', not 'substring/contiguous run'.
- All-equal array $[7,7,7,7,7,7,7] \rightarrow 1$, which rules out 'non-decreasing' (that would give 7) and confirms strictly increasing is required.

Approach / Logic

Classic Longest Increasing Subsequence via $O(N^2)$ dynamic programming. $dp[i]$ holds the length of the longest strictly increasing subsequence ending at index i . For each i , look back at every $j < i$; if $nums[j] < nums[i]$, $dp[i]$ can extend $dp[j]$. The answer is the maximum value in dp .

Solution Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<int> nums(n);
        for (int i = 0; i < n; i++) {
            cin >> nums[i];
        }
        vector<int> dp(n, 1);
        int ans = 1;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) {
```

```
        dp[i] = max(dp[i], dp[j] + 1);
    }
}
ans = max(ans, dp[i]);
}
cout << ans << endl;
}
return 0;
}
```

Question 7: Count Palindromic Substrings

Test Case Description

First line contains an integer T, the number of test cases.

Each of the next T lines contains a string S.

Sample I/O

Sample Input	Sample Output
4	1
b	2
ab	6
abba	7
level	

Difficulty: **Hard**

Observations (how a student can reverse-engineer the logic)

- "b" (single char) → 1: a single character is trivially a palindrome.
- "ab" → 2: only the two single-character substrings 'a' and 'b' are palindromes ("ab" itself is not).
- "abba" → 6: single chars a,b,b,a (4) + "bb" (1) + "abba" (1) = 6, so overlapping/multiple palindromic substrings are all counted, not just the longest one.
- "level" → 7: the 5 single characters + "eve" + "level" = 7, confirming the count includes every palindromic substring of every length.

Approach / Logic

Use the 'expand around center' technique. For every index i, expand outward once treating i as the center of an odd-length palindrome, and once treating the gap between i and i+1 as the center of an even-length palindrome. Each successful expansion (matching characters on both sides) counts one more palindromic substring. Summing counts from every center gives the total.

Solution Code (C++)

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int T;
    cin >> T;
    while (T--) {
        string s;
        cin >> s;
        int n = s.length();
        int count = 0;
        for (int i = 0; i < n; i++) {
            // Odd length palindromes
            int l = i, r = i;
            while (l >= 0 && r < n && s[l] == s[r]) {
                count++;
                l--;
                r++;
            }
            // Even length palindromes
            l = i, r = i + 1;
            while (l >= 0 && r < n && s[l] == s[r]) {
                count++;
                l--;
                r++;
            }
        }
        cout << count << endl;
    }
}
```

```
        r++;
    }
    // Even length palindromes
    l = i;
    r = i + 1;
    while (l >= 0 && r < n && s[l] == s[r]) {
        count++;
        l--;
        r++;
    }
}
cout << count << endl;
}
return 0;
}
```

Question 8: Last Row of Pascal's Triangle

Test Case Description

First line contains an integer T ($1 \leq T \leq 100$), the number of test cases.

The next T lines contain an integer N ($N < 30$), the input value for each test case.

Sample I/O

Sample Input	Sample Output
2 5 1	1 4 6 4 1 1

Difficulty: **Medium**

Observations (how a student can reverse-engineer the logic)

- $N = 5$ gives "1 4 6 4 1" — exactly row 4 (0-indexed) of Pascal's triangle, which has 5 entries: $C(4,0)=1$, $C(4,1)=4$, $C(4,2)=6$, $C(4,3)=4$, $C(4,4)=1$.
- $N = 1$ gives "1" — a single entry, matching row 0 of Pascal's triangle.
- So N doesn't index the row directly by its usual number; rather, N is the number of entries to print, i.e. the $(N-1)$ -th row.

Approach / Logic

Print the $(N-1)$ -th row of Pascal's triangle using the binomial-coefficient recurrence: starting with $val = 1 = C(n-1, 0)$, each next entry is obtained via $val = val * (n-1-i) / (i+1)$, which incrementally computes $C(n-1, i+1)$ from $C(n-1, i)$ without recalculating factorials.

Solution Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        long long val = 1;
        for (int i = 0; i < n; i++) {
            cout << val << " ";
            val = val * (n - 1 - i) / (i + 1);
        }
        cout << endl;
    }
    return 0;
}
```

Question 9: Maximum Points on a Line

Test Case Description

First line contains an integer T, the number of test cases.

The next line contains an integer N, the number of points. The next N lines each contain two integers X and Y, the coordinates of a point.

Sample I/O

Sample Input	Sample Output
2 3 1 1 2 2 3 3 6 1 1 3 2 5 3 4 1 2 3 1 4	3 4

Facilitator note: The sample output was not included with the original source material. It was derived independently by implementing and running the provided code against the given sample input, then cross-checked by hand (all 3 points in the first test case are collinear on $y = x$; the second test case has a max of 4 collinear points, e.g. (1,1), (3,2), (5,3) plus one more on the same line).

Difficulty: Hard

Observations (how a student can reverse-engineer the logic)

- In the first test case, all three points (1,1), (2,2), (3,3) lie on the line $y = x$ — all 3 are collinear, so the answer is the total number of points.
- In the second test case, checking slopes between pairs shows that (1,1), (3,2), (5,3) are collinear (each step moves +2 in x, +1 in y); one more of the remaining points shares that same line, giving a max of 4.
- This points toward a geometry problem: for every point, measure its slope/direction to every other point, and find the largest group of points sharing the same direction from a common point.

Approach / Logic

For each point i , compute the direction (dy , dx) to every other point j , reduced to lowest terms via GCD and normalized to a canonical sign so that opposite-direction vectors are treated the same. Points sharing the same (dy , dx) direction from point i are collinear with it. Track counts per direction in a map, add back any duplicate (identical) points, and take the maximum group size found across all choices of i .

Solution Code (C++)

```
#include <vector>
#include <map>
#include <numeric>
#include <algorithm>
using namespace std;
```

```

class Solution {
public:
    int maxPoints(vector<vector<int>>& points) {
        int n = points.size();
        if (n <= 2)
            return n;
        int ans = 0;
        for (int i = 0; i < n; i++) {
            map<pair<int, int>, int> mp;
            int duplicates = 1; // count point i itself
            int currMax = 0;
            for (int j = i + 1; j < n; j++) {
                int dy = points[j][1] - points[i][1];
                int dx = points[j][0] - points[i][0];
                // Handle identical points
                if (dx == 0 && dy == 0) {
                    duplicates++;
                    continue;
                }
                int g = std::gcd(abs(dy), abs(dx));
                dy /= g;
                dx /= g;
                // Enforce canonical orientation: dx > 0, or (dx == 0 && dy > 0)
                if (dx < 0 || (dx == 0 && dy < 0)) {
                    dx = -dx;
                    dy = -dy;
                }
                mp[{dy, dx}]++;
                currMax = max(currMax, mp[{dy, dx}]);
            }
            ans = max(ans, currMax + duplicates);
        }
        return ans;
    }
};

```